GraphIoT: Lightweight IoT Device Detection Based on Graph Classifiers and Incremental Learning

Yansong Yin, Kun Xie, *Member, IEEE*, Shiming He, Yanbiao Li, *Member, IEEE*, Jigang Wen, Zulong Diao, Dafang Zhang, and Gaogang Xie

***Abstract*—The rapid expansion of the Internet of Things (IoT) has led to growing concerns about the security of IoT devices. A crucial aspect of ensuring their security is IoT device identification, which involves pinpointing the specific type of device. Existing solutions, however, either necessitate complex feature engineering or struggle to handle the ever-increasing number of new devices in open IoT environments. To tackle these challenges, this paper introduces GraphIoT, a lightweight IoT device detection method based on graph classifiers. GraphIoT leverages lightweight flow information, such as packet length, direction, and timestamp, to create an IoT Device Traffic Graph Representation (IoT-DTGR). This representation offers a comprehensive view of IoT device flows while preserving features in bidirectional IoT Device-Gateway interactions. By transforming the IoT device detection problem into a graph classification problem, GraphIoT employs a powerful Graph Neural Network that takes into account both node and edge features, as well as subgraph structures in IoT-DTGRs, to classify graphs and consequently identify device types. Additionally, the paper proposes an incremental learning framework called CL-GraphIoT that continuously learns features of new IoT device flows without forgetting previously learned device features. This is achieved through two strategies: parameter sharing and sample replaying. The paper gathers a real-world dataset from 18 IoT devices and conducts experiments on two datasets: the gathered real-world dataset and an open-source dataset covering 21 IoT device types. The experimental results demonstrate that both GraphIoT and CL-GraphIoT outperform state-of-the-art methods, achieving high accuracy in device detection with fast processing speed.**

***Index Terms*—Graph neural networks, incremental learning, IoT device detection.**

I. Introduction

A. Motivation and Background

THE Internet of Things (IoT) has dramatically changed how we interact with technology, integrating it into everyday objects such as cameras, lights, and appliances. As depicted in Fig. 1, a typical smart home today consists of multiple interconnected IoT devices. Estimates suggest that by 2025, approximately 7.544 billion IoT devices will be in use [1]. With the rapid proliferation of these devices, effective device detection has become essential for ensuring Quality of Service (QoS), enhancing security through intrusion detection, and maintaining robust prevention systems.

However, the detection of IoT devices presents significant challenges due to the diversity of device types, communication protocols, and control interfaces. The continual emergence of new device types further complicates the detection process in ever-expanding IoT environments.

B. State of the Art in IoT Device Identification

Recent advancements in IoT device identification have led to the development of two primary types of identification algorithms: rule-based methods and machine learning-based methods. The machine learning-based category is further divided into traditional machine learning methods and deep learning methods.

Rule-based methods [2], [3], [4], [5], [6], [7], [8], [9] rely on predefined rules linking device types to basic features such as server IP addresses and domain names. Although these methods are computationally efficient, they struggle with new and unknown devices and often cannot differentiate between devices from the same manufacturer using the same servers.

Traditional machine learning-based methods [10], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24] mainly include techniques such as support vector machines, random forests, decision trees, and clustering, which classify device types based on features extracted from network traffic. Despite their effectiveness, these methods are heavily reliant on complex feature engineering, which is both resource-intensive and time-consuming. Furthermore, they do not adequately address the need for continuous learning to accommodate new types of IoT devices.

Received 19 April 2024; revised 18 August 2024; accepted 19 September 2024. Date of publication 24 September 2024; date of current version 30 December 2024. The work was supported in part by the National Natural Science Foundation of China under Grant 62025201, under Grant 62321166652, under Grant 62472167 and under Grant 62272062, in part by the Hunan Provincial Natural Science Foundation of China under Grant 2024]J3014 and under Grant 2024]J5165, in part by the Science and Technology Innovation Program of Hunan Province under Grant 2023RC3139, and in part by the Key Research and Development Program of Hunan Province under Grant 2023GK2001. *(Corresponding author: Kun Xie.)*

Yansong Yin and Kun Xie are with the College of Computer Science, Electronics Engineering, Hunan University, Hunan 410012, China, and also with the Ministry of Education Key Laboratory of "Fusion Computing of Supercomputing, Artificial Intelligence", Guangzhou 510000, China (e-mail: xiekun@hnu.edu.cn).

Shiming He is with the School of Computer Science, Communication Engineering, Changsha University of Science, Technology, Hunan 410205, China.

Yanbiao Li and Gaogang Xie are with the Computer Network Information Center, Chinese Academy of Sciences, Beijing 100045, China, and also with the School of Computer Science, Technology, University of the Chinese Academy of Sciences, Beijing 101408, China.

Jigang Wen is with the School of Computer Science, Engineering, Hunan University of Science, Technology, Xiangtan 411199, China.

Zulong Diao is with the Institute of Computing Technology, Chinese Academy of Sciences, Beijing 100045, China, and also with the Purple Mountain Laboratories, Nanjing 210023, China.

Dafang Zhang is with the College of Computer Science, Electronics Engineering, Hunan University, Hunan 410012, China.

Digital Object Identifier 10.1109/TSC.2024.3466854

1939-1374 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Missouri Libraries. Downloaded on August 13,2026 at 05:55:03 UTC from IEEE Xplore. Restrictions apply.

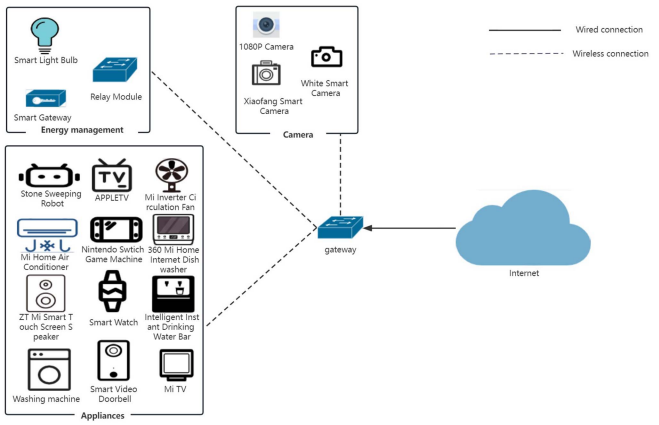


Fig. 1. Smart home environment.

Deep learning-based methods [11], [25], [26], [27], [28], [29], [30], [31], [32], [33], such as CNNs and RNNs, have shown promise in their ability to autonomously learn and extract intricate features from raw data. However, they typically suffer from slow identification speeds and high response delays, and they struggle to adapt to new devices in dynamic IoT environments.

The primary challenge in IoT device detection is developing a method that can quickly adapt to new device types without relying on extensive manual feature engineering. There is a pressing need for a lightweight, adaptable identification algorithm capable of handling the dynamic and continuously evolving landscape of IoT devices. Such an algorithm would not only improve detection speeds but also enhance the scalability and flexibility of IoT security systems.

C. Our Technique and Contributions

To tackle the intricacies of IoT device detection, we introduce a novel, lightweight framework named GraphIoT, which employs an innovative approach known as IoT Device Traffic Graph Representation (IoT-DTGR). Our inspiration stems from the observation that graph structures generated from different devices exhibit unique characteristics. Specifically, GraphIoT transforms device flows into graph structures using minimal flow information, such as packet length, direction, ordering, and timestamps. This approach not only retains the comprehensive details of the original flow but also positions the device detection problem within the framework of graph classification.

GraphIoT incorporates a Graph Neural Network (GNN)-based classifier that efficiently extracts relevant features from nodes and edges, as well as subgraph patterns, directly from the IoT-DTGR inputs. This capability significantly enhances the accuracy of identifying IoT devices under real-world conditions.

Extending our framework, we also introduce CL-GraphIoT, an incremental learning model that adapts to new IoT device flows while retaining knowledge of previously learned devices. This model is engineered to be lightweight, requiring only three GNN layers, thus ensuring rapid detection speeds and low computational complexity. Our technical contributions are summarized as follows:

- *Innovative Representation:* We propose IoT-DTGR, a novel method for representing IoT device traffic as graphs. Each flow is depicted as a graph where vertices represent packets, and edges denote interactions between IoT devices and the gateway at the packet level. This representation significantly reduces the computational burden associated with complex feature engineering in traditional machine learning-based device detection.
- *Graph Neural Network Classifier:* We develop GraphIoT, a GNN-based classifier for IoT device detection. This classifier leverages node and edge features along with subgraph structures within IoT-DTGRs to differentiate between device flows, marking the first application of GNNs in this domain.
- *Continuous Learning Framework:* We propose CL-GraphIoT, which incorporates strategies of parameter sharing and sample replaying to facilitate ongoing learning of new device types. This framework ensures that the system can incorporate new IoT devices without complete retraining, a first in the field using GNNs in conjunction with incremental learning.
- *Empirical Validation:* We evaluate our methods using a real-world dataset collected from 18 IoT devices, as well as an open-source dataset encompassing 21 device types. Our approaches, GraphIoT and CL-GraphIoT, outperform existing methods in terms of classification accuracy and demonstrate superior continuous learning capabilities in dynamic IoT environments.

The structure of the paper is organized as follows. Section II reviews related work. Section III details the problem definition, the graphical model, and provides an overview of our solution. Section IV introduces the graph learning and graph classification modules. Section V elaborates on our adaptive framework for incremental learning. Section VI discusses the experimental setup and results, comparing our approach with existing methods. We conclude the paper in Section VII.

II. Related Work

IoT device detection techniques have evolved significantly, encompassing rule-based and machine learning-based approaches. The machine learning-based category further includes traditional machine learning methods and deep learning methods.

A. Rule-Based Methods

Rule-based approaches often rely on unique device behavior or network fingerprints. Gao et al. [4] utilized wavelet analysis to identify devices based on packet modifications specific to routing devices. However, their method does not extend to endpoint IoT devices, which are our primary focus. Guo and Heidemann [8] identified IoT devices by monitoring periodic connections using server names and IP addresses, but this approach is prone to collisions for devices from the same manufacturer. Swarnkar et al. [9] developed BitIoT, a novel unsupervised method using Deep Packet Inspection and BinMap for device classification.

Despite their utility, rule-based methods are limited by the need to track numerous dynamic IP addresses and maintain updated rules, making them less effective against new and unknown devices.

Authorized licensed use limited to: University of Missouri Libraries. Downloaded on August 13,2026 at 05:55:03 UTC from IEEE Xplore. Restrictions apply.

B. Machine Learning-Based Methods

1) Traditional Machine Learning-Based Methods: Traditional machine learning-based methods have predominantly utilized supervised learning techniques [10], [13], [15], [16], [17], [19], [20], [21], [22], [24], with fewer implementations using unsupervised learning [12], [14], [18], [23]. These methods typically involve extensive feature engineering, which entails labor-intensive data collection and processing. For example, Miettinen et al. employed packet header analysis with a Random Forest classifier in their IoT Sentinel system [15]. It utilized eight statistical values as features, including the link-layer protocol, network-layer protocol, transport-layer protocol, application-layer protocol, IP options, packet length, IP address, and port number. AUDI introduced by Samuel et al. [12] and the multi-stage classification algorithm proposed by Sivanathan et al. [19] are notable examples. AUDI [12] relied on 33 statistical features and employed a supervised K-Nearest Neighbors (KNN) classifier for automatic device type recognition. Sivanathan et al. [19] utilized 13 statistical features and established a multi-stage classifier for IoT device identification, incorporating both Naive Bayes Multinomial and Random Forest classifiers.

Despite achieving high accuracy, traditional machine learning methods heavily rely on complex feature engineering, which requires extensive data handling and computational resources. Additionally, they face challenges in adapting to new types of IoT devices through continuous learning.

2) Deep Learning-Based Methods: Deep learning has shown promising results in various domains [34], [35], [36] but its application in IoT device detection remains in early stages. Lopez-Martin et al. [25] combined Convolutional and Recurrent Neural Networks for traffic classification, which, despite its effectiveness, suffers from long training times. Jorge Ortiz et al. [26] employed an LSTM-based self-encoder for packet sequence representation, and Bao et al. [11] introduced a hybrid framework combining deep neural networks with clustering for device classification. Chowdhury et al. [27] used CNNs to extract features for device classification, and HSGAN-IoT [28] integrated a semi-supervised GAN to improve classification performance. Luo et al. [30] proposed a method for identifying IoT device types based on Transformers, and Rey et al. [31] proposed a framework that used joint learning to detect malware affecting IoT devices. Fan et al. [32] proposed a novel IoT device identification model called AutoIoT, which uses convolutional neural networks and fully connected layers to detect IoT devices, and Zahid et al.[33] proposed a robust framework based on hierarchical deep neural networks for identifying and classifying IoT devices. Solatidehkordi et al. [29] proposed a smart home appliance classification system that leverages the deep learning architecture of Long Short-Term Memory (LSTM) networks.

Deep learning approaches, however, often involve complex architectures requiring extensive training and long inferring, and are less suited for dynamic, real-time IoT environments due to their inability to rapidly adapt to new device types.

III. Problem Definition, Graphical Model, and Solution Overview

A. Problem Definition

In this work, we address the challenge of categorizing Internet of Things (IoT) device traffic into distinct types based on lightweight flow characteristics. Unlike conventional methods that rely on capturing comprehensive packet data at the IoT gateway and calculating intricate statistical features, our approach utilizes minimal information-specifically, the lengths and timestamps of upstream and downstream packets.

During the interaction between IoT devices and gateways, a 'device flow' refers to a series of related data packets transmitted sequentially between the source MAC address (IoT device) and the destination MAC address (gateway). Since this interaction does not rely on specific protocols and ports, the types of data flows are not restricted. This means that the flows can be TCP flows, UDP flows, or other protocols data flows. Information such as packet length, timestamp, and direction is exclusively collected at the gateway.

We primarily focus on the types of IoT devices in smart homes. In this scenario, a single gateway usually interacts with the IoT devices, and interactions involving multiple gateways are rare. However, in networks such as those of drones, multiple gateways may interact with IoT devices. Each gateway captures data such as packet length, timestamp, and direction, and is responsible for collecting communication data from the devices within its coverage area. The data collected by all gateways is then aggregated and processed.

Our objective is to precisely determine the type of IoT device from an analysis of a sequence of *COUNT* consecutive packets sent to or received from the device. Each packet within this sequence, denoted as the i-th packet, is represented by a triplet $x_i = [d_i, l_i, t_i]$, where d_i indicates the packet's direction, l_i its length, and t_i its timestamp. An IoT device's traffic flow is then encapsulated by a series of *COUNT* such packets, expressed as $iotd_n = \{x_1^{(n)}, x_2^{(n)}, \ldots, x_{COUNT}^{(n)}\}$, where $x_i^{(n)}$ is the i-th packet in the n-th flow from the device. Our goal is to devise an efficient algorithm that leverages this lightweight flow data ($iotd_n$) to effectively classify IoT devices.

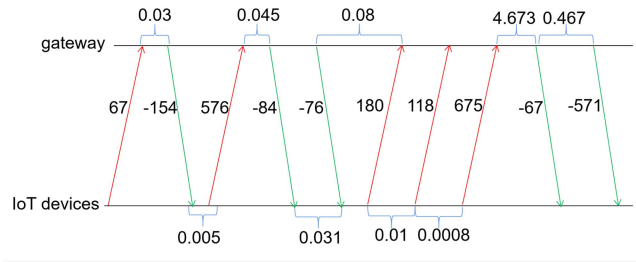
B. Graph Models of IoT Devices

The behavior of IoT devices is influenced by the protocols they adhere to, which dictate the nature of their interactions. To effectively analyze these interactions, we aim to construct graph models that encapsulate the processes involved. These models have the potential to facilitate the identification and differentiation of IoT devices based on the unique characteristics of their interaction graphs.

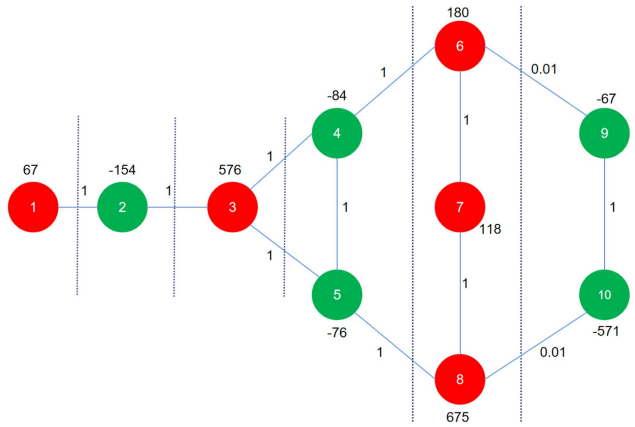
1) Interactions Between an IoT Device and its Gateway: The packet-level interactions between an IoT device and its gateway are visualized in Fig. 2(a). This diagram employs two parallel lines to symbolize the IoT gateway and the IoT device, respectively.

In the diagram, each directed arrow signifies a packet transfer, with the arrow's label indicating the packet's length. Symbols on the arrows denote the direction of packet flow: "+" for upstream (red arrows, sent from the device to the gateway) and "−" for downstream (green arrows, sent from the gateway to the device). For instance, in the presented scenario, the IoT device initiates a sequence of ten packets ($COUNT = 10$). The first packet, represented by an upstream red arrow, has a length of 67, indicating it is sent from the device to the gateway. Conversely, the second packet, denoted by a downstream green arrow, has a length of -154, indicating transmission from the gateway to the device. The temporal gap between consecutive packets is depicted by the numerical value placed between the arrows; in

Authorized licensed use limited to: University of Missouri Libraries. Downloaded on August 13,2026 at 05:55:03 UTC from IEEE Xplore. Restrictions apply.



(a) Packet-level IoT Device to Gateway Interaction Diagram.



(b) Graphical Representation of IoT Device Traffic (IoT-DTGR).

Fig. 2. Graph-based models of IoT device interactions.

this example, a 0.03-second interval separates the initial pair of packets.

2) IoT Device Traffic Graph Representation (IoT-DTGR): To better understand the intricacies of IoT device interactions as illustrated in Fig. 2(a), we introduce a novel graph-based framework, termed IoT Device Traffic Graph Representation (IoT-DTGR), visualized in Fig. 2(b). IoT-DTGR aims to encapsulate the communication patterns of IoT devices, thereby enhancing graph learning applications. The construction of IoT-DTGR is governed by the following set of rules:

a) Vertices: Each vertex in IoT-DTGR represents an individual packet within an IoT device's traffic flow (refer to Fig. 2(b)). Vertices are color-coded to indicate the packet's direction: red for upstream packets and green for downstream packets. Additionally, vertices are annotated with sequential numbers denoting their order within the flow. Adjacent to each vertex, a numeric label indicates the packet's size, with positive values assigned to upstream and negative values to downstream packets.

To systematically organize the vertices, we layer them according to packet direction. Consequently, vertices within the same layer exhibit identical flow direction. For instance, vertices 4 and 5, both downstream packets, are aligned within a single layer, while vertices 1 and 2 are situated in separate layers due to their differing directions.

b) Edges: In IoT-DTGR, edges are categorized into intra-layer and inter-layer types:

- *Intra-layer edges* link vertices sharing the same layer, preserving the temporal order of packet arrival. For example, vertices representing packets 6, 7, and 8 are connected in sequence by intra-layer edges 6-7 and 7-8, reflecting their arrival order.

- *Inter-layer edges* bridge vertices from subsequent layers. They are constructed by connecting the first vertex in one layer to the first in the following layer, and similarly, the last vertex to the last in the subsequent layer. For instance, edge 6-9 connects the initial vertices of two consecutive layers, whereas edge 8-10 links the terminal vertices.

In cases where a layer contains a single vertex, it is treated as both the initial and terminal vertex of that layer, resulting in two inter-layer edges to the corresponding vertices of the adjacent layer. For example, edges 3-4 and 3-5 connect the lone vertex 3 to vertices 4 and 5, the initial and terminal vertices of the subsequent layer, respectively.

It is noteworthy that if two consecutive layers each contain only one vertex, a single edge is established between them, as demonstrated by edge 1-2 between the solitary vertices of the first and second layers.

c) Edge Weights: The temporal relationship between packets is captured by the edge weights in IoT-DTGR. Recognizing that different IoT devices may exhibit distinctive timing patterns, we leverage these temporal signatures for device differentiation.

Nevertheless, network environmental factors such as congestion and interference might alter packet timing, potentially compromising the distinctiveness of IoT-DTGR. To mitigate this, we classify time intervals into 'long-term' (greater than 1 second, encoded as 0.01) and 'short-term' (1 second or less, encoded as 1). This categorization assists in maintaining the reliability of IoT-DTGR under varying network conditions.

C. Overview of the Solution: Graph Neural Network Architecture

Upon constructing the IoT-DTGR, we observe that different IoT device categories manifest distinct graph topologies, as illustrated in Fig. 3. This observation propels us to develop a graph-based methodology for the classification of IoT devices.

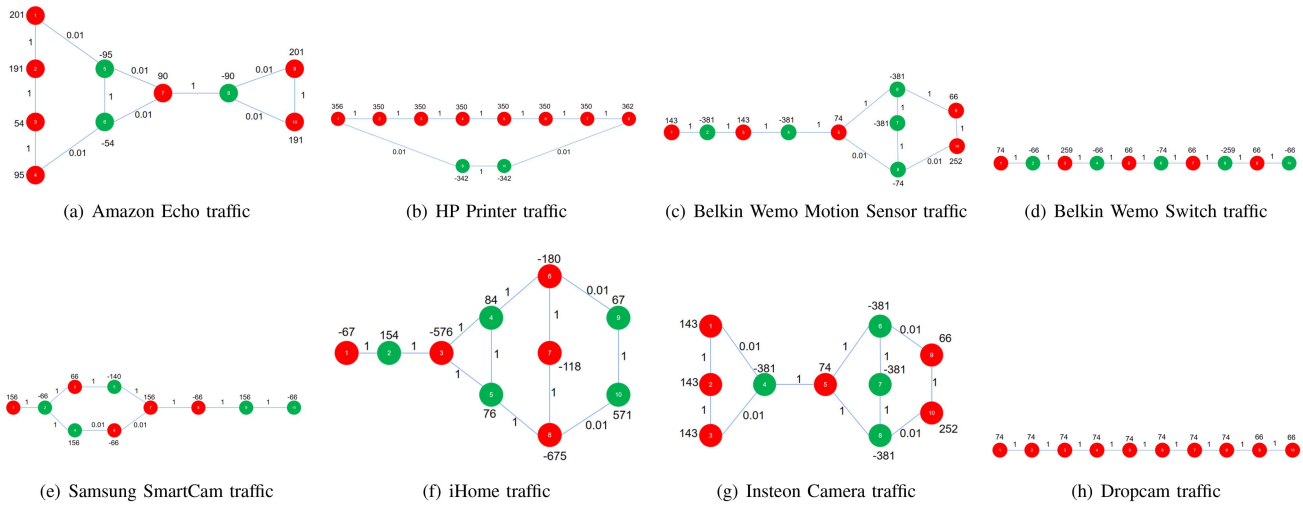
We introduce GraphIoT, a graph neural network (GNN) architecture tailored for precise identification of IoT devices. Fig. 4 depicts the schematic of GraphIoT, which is comprised of three key modules:

- *Graph Construction Module* (highlighted in pink): Transforms the traffic flows of IoT devices ($iotds$) into the IoT Device Traffic Graph Representation (*IoT-DTGR*), adhering to the principles delineated in Section III-B2.
- *Graph Learning Module* (highlighted in green): Learns the graph representations ($h_{IoT-DTGR}$) from the *IoT-DTGR*s, enabling the differentiation of various graph structures to identify IoT device types.
- *Graph Classification Module* (highlighted in orange): Consists of a linear layer followed by a *Softmax* activation function. It leverages the representation vector $h_{IoT-DTGR}$, learned by the graph learning module, to predict the classification of the IoT-DTGRs.

IV. Graph Learning and Graph Classification Modules

This section introduces the graph learning module and the graph classification module, which are essential for a straightforward IoT device identification system based on Graph Neural Networks (GNNs).

Authorized licensed use limited to: University of Missouri Libraries. Downloaded on August 13,2026 at 05:55:03 UTC from IEEE Xplore. Restrictions apply.



(a) Amazon Echo traffic (b) HP Printer traffic (c) Belkin Wemo Motion Sensor traffic (d) Belkin Wemo Switch traffic

(e) Samsung SmartCam traffic (f) iHome traffic (g) Insteon Camera traffic (h) Dropcam traffic

Fig. 3. Sample IoT traffic graphs for eight different device types.

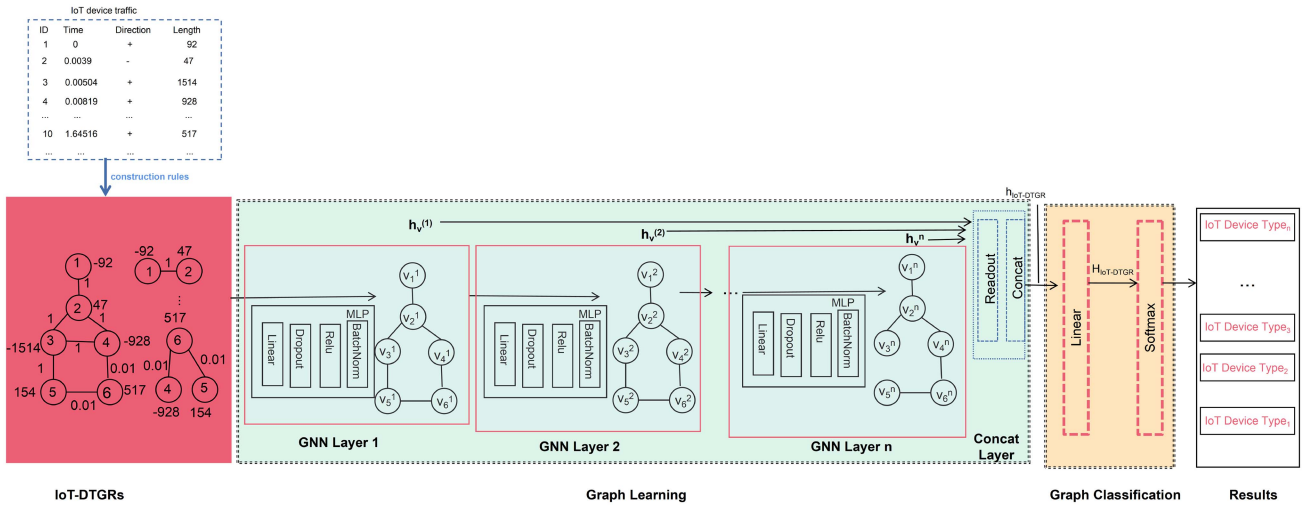


Fig. 4. Structural overview of GraphIoT.

A. *Straightforward IoT Device Identification Based on GNNs*

Graph algorithms, and in particular the Weisfeiler-Lehman (WL) test [37], are instrumental for discerning differences between graph structures. This test iterates over vertices, updating their information based on the data from one-hop neighbors. Message-passing GNNs, as demonstrated by Xu et al. [38], emulate this procedure to differentiate between graph topologies. Leveraging this concept, we outline a straightforward two-step IoT device identification algorithm using GNNs:

Step 1. Update Vertex Information: The information of each vertex is updated iteratively through a process called neighborhood aggregation. The vertices exchange and update their representation vectors with adjacent vertices as follows:

$$h_v^{(k)} = \phi\left(h_v^{(k-1)}, f\left(\left\{h_u^{(k-1)} : u \in \mathcal{N}(v)\right\}\right)\right) \quad (1)$$

In (1), $h_v^{(k)}$ denotes the representation vector of vertex v at the k-th iteration. This vector is updated by the functions $\phi(\cdot)$ and $f(\cdot)$, where $\phi(\cdot)$ combines the vertex's previous information $h_v^{(k-1)}$ with the aggregated data from its neighbors $\mathcal{N}(v)$, processed through the function $f(\cdot)$.

Step 2. Construct the Graph Representation Vector: After n iterations of information updates, the graph's representation vector is assembled by concatenating the updated vertex representation vectors. This comprehensive vector is then utilized in a classification network to identify the type of IoT device.

B. *Limitations of the Straightforward Solution*

The WL test, as well as message-passing GNNs, rely on iterative aggregation of information from immediate neighbors, expanding with each iteration to include 2-hop, 3-hop, and further neighbors. However, certain structurally distinct graphs, specifically IoT-DTGR$_1$ and IoT-DTGR$_2$ (depicted in Figs. 5(a) and 6(a)), are both 4-regular and cannot be distinguished by these methods.

Authorized licensed use limited to: University of Missouri Libraries. Downloaded on August 13,2026 at 05:55:03 UTC from IEEE Xplore. Restrictions apply.

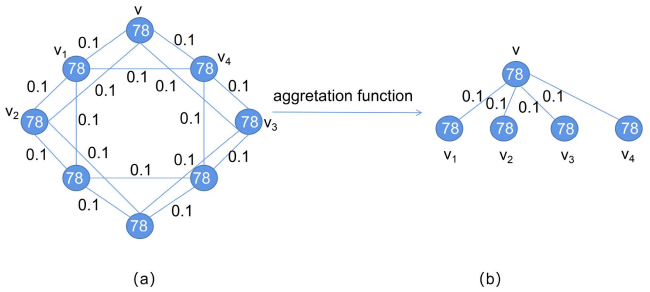


Fig. 5. The structure of IoT-DTGR$_1$.

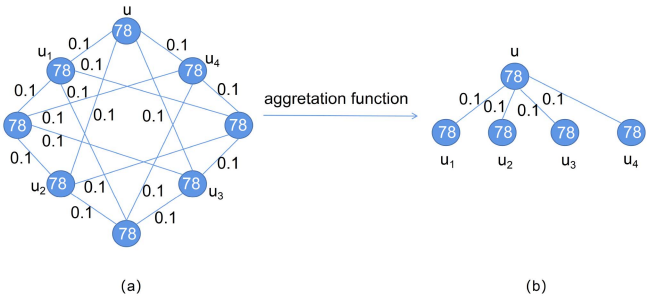


Fig. 6. The structure of IoT-DTGR$_2$.

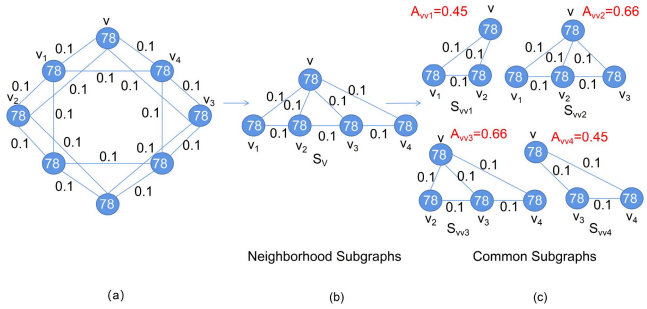


Fig. 7. One-hop neighbor expansion in IoT-DTGR$_1$.

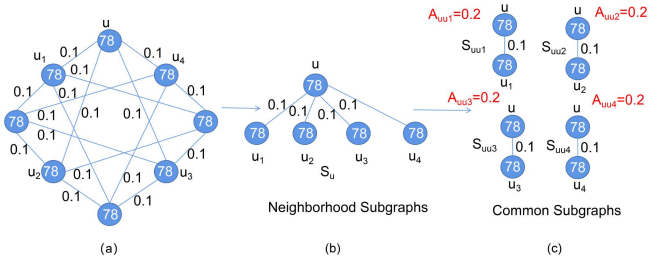


Fig. 8. One-hop neighbor expansion in IoT-DTGR$_2$.

As demonstrated in Figs. 5(b) and 6(b), the one-hop neighbor expansions for each node in IoT-DTGR$_1$ and IoT-DTGR$_2$ are indistinguishable. Hence, when the WL test is applied, the aggregation and update process leads to identical information for both graphs, rendering it ineffective for differentiating between them.

C. Advancing IoT Device Identification via Subgraph-Enhanced Aggregation Strategies

The Weisfeiler-Lehman (WL) test and traditional message-passing Graph Neural Networks (GNNs) often struggle to effectively discern between graph structures due to the inherent limitations in their aggregation mechanisms. The standard aggregation function $f(\{h_u^{(k-1)} : u \in \mathcal{N}(v)\})$ fails to account for the complex structural nuances among neighboring vertices, leading to reduced discriminative strength [39].

Aiming to rectify this shortcoming and enrich the graph learning process, our proposed framework introduces an innovative approach that integrates subgraph structures into the aggregation function, as formalized in (2):

$$h_v^{(k)} = \phi\left(\{A_{vu}, e_{vu} | u \in \mathcal{N}(v)\}\, h_v^{(k-1)},\right.$$
$$\left. f\left(\left\{A_{vu}, e_{vu}, h_u^{(k-1)} | u \in \mathcal{N}(v)\right\}\right)\right) \tag{2}$$

In the equation above, $h_v^{(k)}$ signifies the representation vector of vertex v at the k-th iteration, $\phi(\cdot)$ is used as a general update function, which in this paper is implemented as a trainable Multi-Layer Perceptron (MLP), while $f(\cdot)$ is used as the aggregation function. The neighborhood of vertex v is represented by $\mathcal{N}(v)$, and e_{uv} is the temporal interval of the edge connecting vertices u and v.

The neighbor subgraph S_v is composed of vertex v, its adjacent vertices, and the edges that interconnect them. The shared subgraph S_{vu}, defined as $S_{vu} = S_v \cap S_u$, encapsulates the shared neighboring vertices of v and u and the edges linking them.

The structural coefficients A_{vu} within (2) reflect the intricate connections in the shared subgraph S_{vu}, where A_{vu} aims to encapsulate the structural relationship between vertex v and its neighbor u. This coefficient is mathematically detailed in (3):

$$A_{vu} = \frac{|E_{vu}|}{|V_{vu}| \cdot |V_{vu} - 1|} \cdot |V_{vu}|^2 \tag{3}$$

$$|E_{vu}| = \sum e_{vu} \tag{4}$$

Here, $|E_{vu}|$ is the sum of edge weights in the subgraph S_{vu}, and $|V_{vu}|$ is the count of vertices within the same subgraph.

Through the examination of IoT-DTGR$_1$ and IoT-DTGR$_2$ as presented in Figs. 7(a) and 8(a), we can discern unique neighbor subgraphs (Figs. 7(b) and 8(b)) as well as distinct pairwise common subgraphs (Figs. 7(c) and 8(c)). By factoring in these subgraph structures, our framework is equipped to capture more detailed local graph structures, thereby enhancing the ability to differentiate between graphs that share similar global configurations.

D. Enhanced Graph Learning Module Architecture

Fig. 4 presents the architecture of the Graph Learning module, comprised of a series of Graph Neural Network (GNN) layers followed by a concatenation layer, as shown in Fig. 9. This structure facilitates the iterative refinement of vertex representations through the GNN layers, culminating in a comprehensive graph representation.

Each GNN layer applies the aggregation function from (2) to update vertex representations. The specific formulation used in

Authorized licensed use limited to: University of Missouri Libraries. Downloaded on August 13,2026 at 05:55:03 UTC from IEEE Xplore. Restrictions apply.

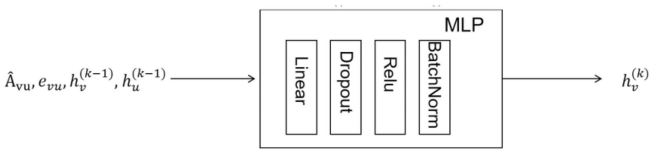


Fig. 9. Each GNN learning layer.

our implementation is outlined in (5):

$$h_v^{(k)} = MLP\left(\epsilon^{(k-1)} \sum_{u \in \mathcal{N}(v)} \left(\hat{A}_{vu} + e_{vu}\right) h_v^{(k-1)} + \sum_{u \in \mathcal{N}(v)} \left(\hat{A}_{vu} + e_{vu}\right) h_u^{(k-1)}\right) \tag{5}$$

In this equation, $h_v^{(k)}$ represents the feature vector of vertex v at the k-th layer, $\mathcal{N}(v)$ denotes the set of neighboring vertices to v, e_{uv} is the time interval of the edge between vertices u and v, ϵ is a learnable parameter, and $\hat{A}vu$ is the normalized adjacency coefficient defined as $\hat{A}vu = \frac{A_{vu}}{\sum_{u \in \mathcal{N}(v)} A_{vu}}$.

The multilayer perceptron (MLP) within (5) is implemented using a sequence of Linear, BatchNorm, Dropout, and ReLU functions, which together form each GNN learning layer, as depicted in Fig. 9.

Following the GNN layers, the concatenation layer assembles the final graph representation by merging the output of a *Readout* function across all layers, generating a graph vector leveraged for the classification of IoT device types. This process is mathematically described in (6):

$$h_{IoT-DTGR} = Concat\left(Readout\left(\left\{h_v^{(k)} | v \in IoT - DTGR\right\}\right) | k \in [0, n]\right) \tag{6}$$

E. Graph Classification Module

In the subsequent phase, the GraphIoT framework transforms the high-dimensional graph vector $h_{IoT-DTGR}$ into a device-type-specific feature vector $H_{IoT-DTGR}$ through a linear transformation. The dimensionality D of the resulting vector corresponds to the number of distinct IoT device types. Subsequently, a *Softmax* function generates the probability distribution $\hat{y}$, which elucidates the likelihood of the IoT-DTGR belonging to each device type, as formalized in (7):

$$\hat{y} = Softmax(H_{IoT-DTGR}) \tag{7}$$

F. Feature Analysis for IoT Device Differentiation

GraphIoT distinguishes among IoT devices through the analysis of various flow features within the IoT-DTGR, including:

1) Packet Direction Information: The sequence of packets encoded in the graph layers reflects the directional flow of information, which is instrumental in device identification.

2) Packet Hierarchy Information: Vertices within the same layer denote packets from identical directions, while vertices across different layers signify packets from divergent directions. This hierarchical structure is characteristic of specific IoT device types and is a key discriminative feature.

3) Packet Interaction Information: The IoT-DTGR graphically represents the sequence of packet transmissions, encapsulating the interactions between IoT devices and network gateways.

4) Packet Length and Timing Information: The sequence of packet lengths and their associated timing are vital classification attributes. Vertex and edge attributes in the IoT-DTGR graph correspond to these packet features.

5) Packet Group Related Features: Each IoT-DTGR contains subgraphs representing packet groups within the flow. Discrepancies in the number of vertices or edges between subgraphs of different IoT-DTGRs can indicate variations in device types. Specifically, if two subgraphs from distinct IoT-DTGRs have a mismatch in the number of vertices ($|V_{S_i}| \neq |V_{S_j}|$), they likely represent different device types. Similarly, if the vertex count matches ($|V_{S_i}| = |V_{S_j}|$) but there is a divergence in the number of edges ($|E_{S_i}| \neq |E_{S_j}|$), this also suggests a distinction in device types. These features related to packet groups play a crucial role in the precise classification of IoT devices.

V. Adaptive Framework for Incremental Learning With New IoT Device Types

In the dynamic landscape of the Internet of Things (IoT), the continuous introduction of novel device types necessitates a resilient identification framework capable of adapting without the need for extensive retraining. Traditional approaches involve training on historical data; however, the integration of new device types into the model poses a challenge due to the potential for significant computational overhead and the risk of catastrophic forgetting.

To mitigate these challenges, we introduce the Continuous Learning GraphIoT (CL-GraphIoT), depicted in Fig. 10. CL-GraphIoT integrates two core methodologies: parameter sharing and controlled sample replay.

- *Parameter Sharing:* In this approach, we selectively retrain a subset of the Graph Neural Network (GNN) layers within the graph learning and classification modules upon the introduction of new devices. By treating multiple GNN layers of the GraphIoT as fixed feature extractors, CL-GraphIoT retains these layers, leveraging their learned representations to facilitate the recognition of new device types.
- *Sample Replay:* To prevent the loss of previously acquired knowledge, known as catastrophic forgetting, we implement a sample replay strategy. This involves the retention and subsequent retraining of a representative subset of historical data alongside the data of the newly introduced device types.

The CL-GraphIoT algorithm unfolds in two primary stages:

1) Initially, GraphIoT is employed to establish a baseline by training on the available IoT Device Traffic Graph Representations (IoT-DTGRs) for existing device types.
2) Subsequently, as new device types are introduced, we selectively retrain only the leading GNN layers and the Graph Classification Module. The remaining layers are inherited from the base network, ensuring continuity and preserving the integrity of the previously learned features.

Authorized licensed use limited to: University of Missouri Libraries. Downloaded on August 13,2026 at 05:55:03 UTC from IEEE Xplore. Restrictions apply.

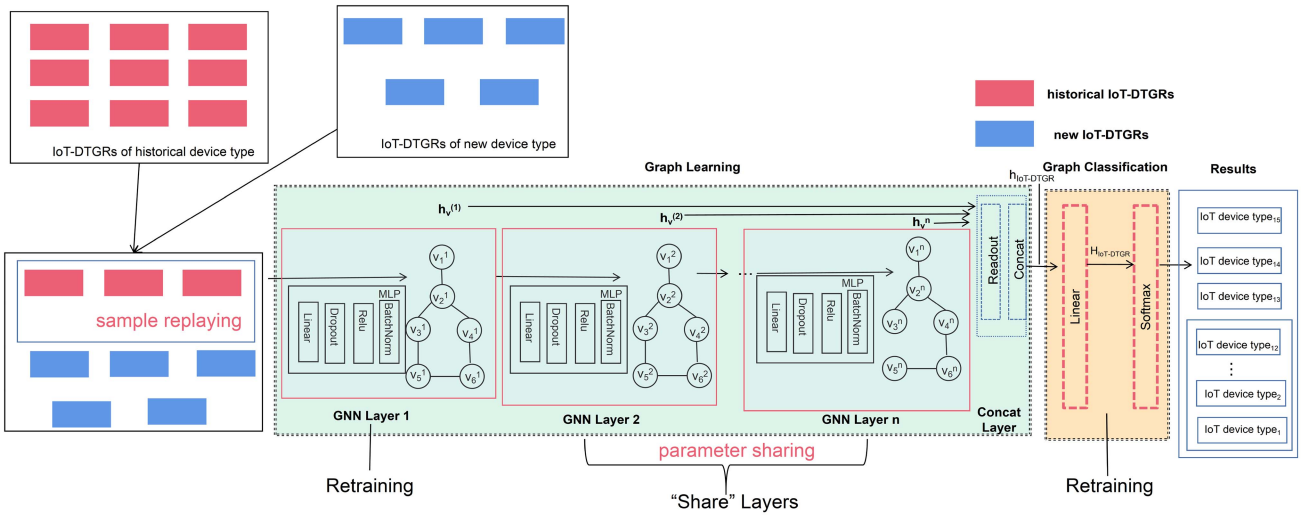


Fig. 10. The CL-GraphIoT framework: Enabling incremental learning within the GraphIoT ecosystem.

TABLE I
DEVICE GROUP COMPOSITION FOR TMC_{21} DATASET

Group 1	Group 2	Group 3	Group 4
Amazon Echo	PIX-STAR Photo-frame	TP-Link Day Night Cloud Camera	Withings Aura Smart Sleep Sensor
Belkin Wemo Motion Sensor	Samsung SmartCam	TP-Link Smart plug	Withings Smart Baby Monitor
Belkin Wemo Switch	Smart Things	Triby Speaker	Withings Smart Scale
HP Printer			
iHome			
Insteon Camera			
Dropcam			
Light Bilbs LiFX Smart Bulb			
Nest Dropcam			
NEST Protect Smoke Alarm			
Netatmo Weather Station			
Netatmo Welcome			

During the retraining phase, we judiciously incorporate data from the new device types and a carefully chosen subset of historical data (*either* "Before" *or* "Random" number of instances from each previously learned device type). This strategy is designed to optimize training efficiency, conserve resources, and circumvent the pitfalls of catastrophic forgetting.

VI. EVALUATION

We conduct extensive experiments to address key questions for our GraphIoT and CL-GraphIoT frameworks: For GraphIoT: 1) How do the number of packets and the features used affect accuracy and performance? 2) How does the number of GNN layers influence accuracy, training time, and testing time? 3) Can GraphIoT achieve higher accuracy and faster recognition speed than other methods? For CL-GraphIoT: 1)How do the shared GNN layers and the frequency of sample replays impact accuracy and training time? 2) What is the effectiveness of parameter sharing and sample replaying in enhancing performance? 3) How do individual components within CL-GraphIoT and GraphIoT affect overall performance?

A. Experimental Setting

1) Datasets: To validate our proposed method, we conduct experiments using two different datasets. The first dataset, denoted as TMC_{21}, is derived from a public IoT network traffic dataset [40] and includes data from 21 different types of IoT devices, totaling 5,761,170 packets. The second dataset, labeled IoT_{18}, includes traffic data from 18 unique IoT device types collected by us in a smart home environment set up in a residential area in China. The network traffic is captured using tools installed at the gateway of this environment, totaling 4,399,200 packets.

The datasets are summarized in Tables I and II, and the statistical details are provided. The devices from each dataset are categorized into four groups: g_1, g_2, g_3, and g_4 for an experiment conducted in an open IoT environment.

The datasets are randomly divided into 10 subsets of roughly equal size. Out of these subsets, nine are utilized for training purposes, while the remaining subset functions as the test set.

2) Hyper-Parameter Optimization: The process of hyper-parameter optimization is crucial for maximizing the effectiveness of our models. We employ a rigorous method to identify optimal hyper-parameters by initially exploring a predefined search range for each parameter. We select the top-performing k values from this range as potential candidates. Following this, we execute a detailed search across the combinations of these candidates to determine the set of hyper-parameters that delivers the best performance. The search ranges, along with the selected

Authorized licensed use limited to: University of Missouri Libraries. Downloaded on August 13,2026 at 05:55:03 UTC from IEEE Xplore. Restrictions apply.

TABLE II
DEVICE GROUP COMPOSITION FOR IoT_{18} DATASET

Group 1	Group 2	Group 3	Group 4
Relay Module	APPLETV	360 Mi Home Internet Dishwasher	Washing machine
Smart Watch	Nintendo Swtich Game Machine	Mi Inverter Circulation Fan	Smart Video Doorbell
Mi Home Air Conditioner	Xiaofang Smart Camera	Intelligent Instant Drinking Water Bar	Mi TV
Smart Light Bulb			
1080P Camera			
Stone Sweeping Robot			
Smart Gateway			
ZT Mi Smart Touch Screen Speaker			
White Smart Camera			

TABLE III
HYPER-PARAMETER SELECTION FOR GRAPHIOT AND CL-GRAPHIOT

Hyper-parameter	Search Range	Selected Value
Optimizer	[Adam, SGD]	Adam
Learning Rate	[0.001, ... ,0.009]	0.0025
Training Epochs	[5, ... ,10]	10
Batch Size	[32, ... ,256]	64
Activation Functions	[Tanh, Relu]	Relu
Dropout	[0, ... , 0.3]	0.25
Number of GNN Layers	[1, ... , 4]	3
Number of Hidden Unit	[32, ... , 256]	64
Replay Method	[Before, Random]	Before
Shared Layers of GNN	[1, 2, 3, ... ,1-3]	2-3

hyper-parameters for the GraphIoT and CL-GraphIoT models, are presented in Table III.

3) Evaluation Metrics: To assess the performance of our classifier for IoT device flow classification, we utilize a set of metrics that measure the accuracy and reliability of our model. These metrics are essential for understanding both the effectiveness and the potential limitations of our classifier. The definitions of these metrics are as follows:

$$Accuracy(ACC) = \frac{TP + TN}{TP + TN + FP + FN} \tag{8}$$

$$Recall(RC) = \frac{TP}{TP + FN} \tag{9}$$

$$Precision(PR) = \frac{TP}{TP + FP} \tag{10}$$

$$F1 - Score(F1) = 2 \times \frac{PR \times RC}{PR + RC} \tag{11}$$

The terms used in these equations are defined as:

- *True Positive (TP):* Flows correctly identified as belonging to the target class.
- *True Negative (TN):* Flows correctly identified as not belonging to the target class.
- *False Positive (FP):* Flows incorrectly identified as belonging to the target class.
- *False Negative (FN):* Flows incorrectly identified as not belonging to the target class.

B. Experimental Results

1) Validation of Subgraph Structure Effectiveness: Our proposed system, GraphIoT, effectively distinguishes between different types of Internet of Things (IoT) devices by leveraging the IoT Device Traffic Graph Representation (IoT-DTGR). GraphIoT employs a feature fusion strategy that integrates node features, edge features, and subgraph structure features, thereby enhancing the device identification process. To quantify the contribution of each feature set to the identification performance, we conduct experiments with various combinations of these features. The results are summarized in Table IV, providing a comparative analysis. The bold figures in Table IV underscore the highest accuracy achieved for each dataset, illustrating the substantial impact of the comprehensive feature set employed in the GraphIoT framework.

TABLE IV
IMPACT OF FEATURE COMPOSITION ON THE ACCURACY OF GRAPHIOT ACROSS DIFFERENT PACKET COUNTS

Packets	10	15	20	25	30
	TMC_{21} Accuracy (%)				
Node	96.31%	97.05%	97.68%	98.03%	98.62%
Node, Edge	96.58%	97.22%	97.81%	98.14%	98.68%
Node, Sub	97.05%	97.41%	97.95%	98.29%	98.75%
Node, Edge, Sub	97.29%	97.66%	98.15%	98.44%	**98.84%**
	IoT_{18} Accuracy (%)				
Node	89.49%	92.18%	93.52%	94.44%	94.56%
Node, Edge	90.04%	92.80%	94.06%	94.74%	95.07%
Node, Sub	90.19%	93.01%	94.26%	94.93%	95.13%
Node, Edge, Sub	90.65%	93.23%	94.58%	95.14%	**95.17%**

The bold values the highest accuracy.

Initially, the baseline model, which utilizes only node features for device identification, is presented in the second row of Table IV. Our experimental results demonstrate that the integration of all three types of features–node, edge, and subgraph structure–significantly enhances accuracy in device identification across our test datasets. Notably, with a packet count of 30, GraphIoT achieves an accuracy of 98.84% on the TMC_{21} dataset and 95.17% on the IoT_{18} dataset. This performance markedly surpasses that of the models using only node and edge features, highlighting the critical role of subgraph structure features in enhancing the discriminative power of the system.

2) Influence of Packet Count on Performance: The performance of GraphIoT hinges on the number of packets used to build the IoT Device Traffic Graph Representation (IoT-DTGR) for each network flow. Table V illustrates how GraphIoT's accuracy varies with the packet count in individual flows. It is evident that predictive accuracy is positively correlated with the packet count; higher counts typically lead to improved accuracy. Notably, when the packet count reaches 30, GraphIoT demonstrates an accuracy of 98.84% on the TMC_{21} dataset and 95.17% on the IoT_{18} dataset, highlighting the advantage of robust IoT-DTGR in enhancing device type identification. However, it is important to recognize that an increased packet count also results in higher

Authorized licensed use limited to: University of Missouri Libraries. Downloaded on August 13,2026 at 05:55:03 UTC from IEEE Xplore. Restrictions apply.

TABLE V
CORRELATION BETWEEN THE NUMBER OF PACKETS PER FLOW AND THE ACCURACY OF GRAPHIOT

Packets	10	15	20	25	30
	TMC_{21}				
Accuracy (%)	97.29	97.66	98.15	98.44	**98.84**
	IoT_{18}				
Accuracy (%)	90.65	93.23	94.58	95.14	**95.17**

The bold values the highest accuracy.

TABLE VI
IMPACT OF THE NUMBER OF GNN LAYERS ON ACCURACY AND TIME FOR GRAPHIOT

GNN Layers	1	2	3	4
	TMC_{21}			
Accuracy	0.9824	0.9867	**0.9884**	0.9865
Training Time (s)	**1274.11**	1346.57	1396.31	1467.76
Testing Time (ms/device)	**0.37**	0.39	0.41	0.43
	IoT_{18}			
Accuracy	0.9465	0.9499	**0.9517**	0.9503
Training Time (s)	**1006.86**	1059.66	1078.45	1194.89
Testing Time (ms/device)	**0.37**	0.40	0.42	0.44

The bold values the highest accuracy, the shortest training time, and the shortest testing time.

computational demands. Consequently, a balance between accuracy and computational efficiency is established by setting the packet count at 30 for future experiments.

3) Influence of the Number of GNN Layers on Performance: The performance of the GraphIoT system in IoT device detection is influenced by the configuration of its Graph Neural Network (GNN) layers. To explore this, Table VI presents a comprehensive analysis of GraphIoT's performance metrics, including accuracy and both training and testing times across different number of GNN layers.

The analysis reveals that an increase in the number of GNN layers initially enhances the system's accuracy due to the ability to capture more complex features of device behavior. However, the accuracy begins to decline after adding more than three GNN layers, suggesting potential overfitting. Moreover, both training and testing times increase with the addition of each layer, reflecting higher computational overhead.

Considering these findings, a configuration with three GNN layers is determined to be optimal. This choice strikes a balance between achieving high accuracy and maintaining a lightweight, efficient framework suitable for real-time IoT device detection. With this configuration, testing times are notably low, at just 0.41 ms per device for the TMC_{21} dataset and 0.42 ms per device for the IoT_{18} dataset, showcasing the system's capability for rapid deployment.

C. Performance Comparison

1) Accuracy, Recall, and F1 Score Under Different Identification Models: In order to thoroughly evaluate our newly developed model, GraphIoT, we conduct comparative analysis against five methods. These methods are briefly described as follows:

- *Random Forest [15]:* This method utilizes packet length and inter-packet timing intervals as features, employing a Random Forest classifier to identify different types of IoT devices.
- *Decision Tree:* This approach utilizes packet length and inter-packet timing intervals as features, employing a Decision Tree classifier to categorize different types of IoT devices.
- *WL subtree (WL) [37]:* This method is used for graph classification. We apply it to IoT device detection using the constructed IoT-DTGRs. It is based on the Weisfeiler-Lehman iterative encoding algorithm, which captures graph structural features by iteratively updating node features and serves as the foundation for graph kernel methods. The approach compares substructures (subtrees) within graphs, transforms each subtree into its iterative multi-feature encoding, and quantifies the similarity between different graphs by calculating the similarity between these encodings.
- *DeepFingerprinting (CNN) [41]:* This method utilizes packet length and timestamp as features, leveraging Convolutional Neural Networks to construct a robust deep learning model for classifying various IoT device types.
- *LSTM [42]:* This approach leverages packet length and timestamp as features, employing a dual-layer LSTM network followed by a fully connected layer to construct a deep learning model for classifying IoT device types.
- *GAT (Graph Attention Network) [43]:* This method is used for graph classification, employing node labels as node features and utilizing a graph attention mechanism model to learn graph features. The attention mechanism allows the network to dynamically allocate weights to different nodes as needed, thereby better capturing the relationships between nodes. We use the constructed IoT-DTGRs and apply this method by using packet length as node features and subgraph structures, employing the graph attention mechanism to learn graph features for IoT device type detection.
- *GIN [38]:* This method is used for graph classification and employs node labels as node features to learn graph features. We utilize the constructed IoT-DTGRs and apply this method, using only packet length as node features and leveraging graph neural networks to learn graph features for IoT device type detection.

To ensure a fair comparison, all models are implemented using only packet length and timestamp data as features, avoiding complex feature engineering processes.

Table VII presents the performance metrics of various device identification algorithms. Our GraphIoT algorithm achieves superior accuracy, recall, and F1 score compared to other methods. Unlike CNN and LSTM, which analyze sequences of packet lengths and timestamps, GraphIoT redefines device identification as a graph classification task by accepting IoT-DTGR (IoT Device Traffic Graph Representation) as input. This innovative approach allows GraphIoT to make a more nuanced distinction between device types.

When classifying IoT device types using the WL subtree, memory usage significantly increases due to the necessity of storing numerous subtree encodings and their similarity information. Furthermore, the WL subtree exclusively captures the similarity of local substructures, neglecting global structural information, which results in poor classification performance. In contrast, GraphIoT learns features of subgraph structures, nodes, and edges, enabling it to capture not only local substructure features but also the comprehensive global features of the graph, thereby achieving higher classification accuracy.

Authorized licensed use limited to: University of Missouri Libraries. Downloaded on August 13,2026 at 05:55:03 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
PERFORMANCE COMPARISON WITH BASELINE METHODS

Method	DesicionTree	RandomForest	WL	CNN	LSTM	GAT	GIN	GraphIoT
	TMC_{21}							
Accuracy (ACC)	0.8372	0.845	0.8522	0.9747	0.9814	0.9771	0.9862	**0.9884**
Recall (RC)	0.7627	0.7747	0.8522	0.9167	0.9205	0.9184	0.9349	**0.9674**
F1 Score (F1)	0.7596	0.7723	0.8134	0.8989	0.9265	0.9191	0.9306	**0.9637**
	IoT_{18}							
Accuracy (ACC)	0.6557	0.6961	0.7751	0.7302	0.9328	0.9290	0.9456	**0.9517**
Recall (RC)	0.6103	0.6332	0.7751	0.6493	0.8832	0.8392	0.8806	**0.8991**
F1 Score (F1)	0.606	0.6408	0.7521	0.6217	0.8882	0.8514	0.8903	**0.9058**

The bold values the highest accuracy, the highest recall, and the highest F1 score.

TABLE VIII
COMPARISON OF FEATURE EXTRACTION TIME (FET), PREDICTION TIME, AND TOTAL PROCESSING TIME FOR EACH DEVICE ACROSS DIFFERENT IDENTIFICATION METHODS

Method	DesicionTree	RandomForest	WL	CNN	LSTM	GAT	GIN	GraphIoT
	TMC_{21}							
FET (ms/device)	**0.50**	**0.50**	0.52	**0.50**	**0.50**	0.52	0.52	0.52
Prediction Time (ms/device)	0.38	**0.35**	0.40	1.00	0.51	0.45	0.42	0.41
Total Time (ms/device)	0.88	**0.85**	0.92	1.50	1.01	0.97	0.94	0.93
	IoT_{18}							
FET (ms/device)	**0.50**	**0.50**	0.52	**0.50**	**0.50**	0.52	0.52	0.52
Prediction Time (ms/device)	0.38	**0.35**	0.40	1.00	0.51	0.46	0.43	0.42
Total Time (ms/device)	0.88	**0.85**	0.92	1.50	1.01	0.98	0.95	0.94

The bold values the shortest feature extraction time, the shortest prediction time, and the shortest total time.

GIN considers only packet length as the node feature when learning graph features. In contrast, GraphIoT additionally leverages edge and subgraph features, leading to enhanced performance. As a result, GraphIoT outperforms GIN.

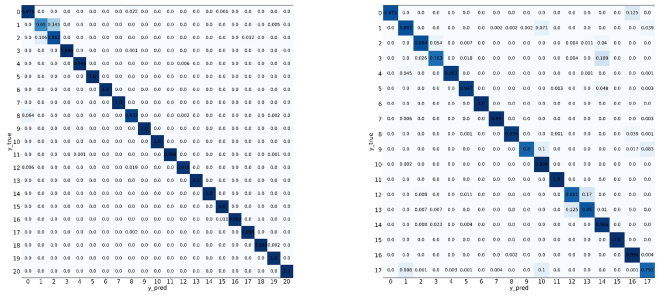
While GAT introduces an attention mechanism for dynamic node weighting, GraphIoT further refines the identification process by precisely defining inter-node relationships and employing graph neural networks to model the entirety of packet flow, thus retaining comprehensive contextual information. Consequently, GraphIoT outperforms GAT.

2) Feature Extraction, Prediction, and Total Time Analysis for Different Identification Models: We assess the time efficiency of various device identification methods by examining three key metrics: Feature Extraction Time (FET), Prediction Time, and Total Time for each device. The results for different methods applied to two datasets, TMC_{21} and IoT_{18}, are detailed in Table VIII.

FET quantifies the time taken to extract critical features, such as packet length and packet timestamp, from the device's network flow and to assemble the graph structure. Prediction Time is defined as the time required for a pre-trained model to analyze test samples and produce outcomes. Total Time is the aggregate duration of both feature extraction and prediction for each device.

While traditional methods like Random Forest and Decision Tree, along with deep learning approaches such as CNN and LSTM, offer faster FETs, GraphIoT, GIN, WL and GAT require slightly more time due to their intricate graph construction. Although all methods deliver prediction results in under 1 millisecond, the complexity of CNN and LSTM leads to longer processing times and slightly reduces accuracy relative to GraphIoT.

Regarding the Total Time, although traditional machine learning methods are marginally quicker, they do not match the accuracy levels achieved by more sophisticated models like GraphIoT. GraphIoT surpasses traditional machine learning methods by approximately 14% in accuracy. Among deep learning techniques, GraphIoT demonstrates the most efficient total time for device identification, outperforming other methods by about 1%.



(a) TMC_{21} dataset (b) IoT_{18} dataset

Fig. 11. Confusion matrix of GraphIoT

Consequently, GraphIoT emerges as a rapid and highly precise method for device identification.

3) Confusion Matrix Analysis of GraphIoT: The performance comparison in Table VII underscores that GraphIoT surpasses all compared methods. To delve deeper into the efficacy of GraphIoT, we present its confusion matrices in Fig. 11 for both the TMC_{21} and IoT_{18} datasets. In each matrix, an entry (i, j) indicates the count of devices belonging to actual device type i but mistakenly identified as type j.

For the TMC_{21} dataset, GraphIoT registers an accuracy exceeding 0.9 for 19 device types and maintains an accuracy over 0.85 for 95% of the IoT devices assessed. In the IoT_{18} dataset, it achieves an accuracy greater than 0.95 for 10 types of IoT devices and exceeds 0.8 accuracy for 89% of the devices.

The robust performance of GraphIoT across various metrics can be attributed to its end-to-end learning architecture. This

Authorized licensed use limited to: University of Missouri Libraries. Downloaded on August 13,2026 at 05:55:03 UTC from IEEE Xplore. Restrictions apply.

TABLE IX
IMPACT OF "SHARE" LAYERS ON ACCURACY AND TIME OF CL-GRAPHIOT

Incremental times	"Share" layer	1	2	3	1,2	2,3	1,2,3
	TMC_{21}						
After g1, g2 comes	Accuracy (ACC)	0.943	0.9442	**0.9467**	0.9295	0.9301	0.9257
	Training Time(s)	104.62	104.73	104.22	102.57	102.01	**95.95**
After g1 and g2, g3 comes	Accuracy (ACC)	0.9305	0.9328	**0.9345**	0.9254	0.9278	0.9185
	Training Time(s)	115.63	115.41	115.74	112.37	112.05	**102.51**
After g1, g2, and g3, g4 comes	Accuracy (ACC)	0.9284	0.9294	**0.93**	0.9223	0.9241	0.9141
	Training Time(s)	154.83	154.24	154.25	149.84	149.67	**131.27**
	IoT_{18}						
After g1, g2 comes	Accuracy (ACC)	0.9807	0.9825	**0.9834**	0.9701	0.9783	0.9703
	Training Time(s)	72.12	72.46	72.34	72	71.84	**70.61**
After g1 and g2, g3 comes	Accuracy (ACC)	0.9607	0.9628	**0.9641**	0.9562	0.9587	0.9484
	Training Time(s)	93.25	93.01	93.54	90.47	90.44	**84.3**
After g1, g2, and g3, g4 comes	Accuracy (ACC)	0.9096	0.9103	**0.9115**	0.9052	0.9086	0.8995
	Training Time(s)	122.86	122.33	122.63	120.89	120.91	**112.58**

The bold values the highest accuracy, and the shortest training time.

architecture enables GraphIoT to directly learn and optimize the feature representation of graphs during classification tasks. Such a mechanism not only enhances the classification accuracy but also provides a deep insight into the node, edge, and subgraph characteristics of the IoT Device Traffic Graph Representation (IoT-DTGR). Consequently, it allows for a more effective differentiation among various IoT device types.

D. Performance Evaluation of CL-GraphIoT

To simulate a real-world open IoT environment, we divide the IoT device types in each dataset into 4 groups. In Tables I and II, we denote the group ID (g_1, g_2, g_3, and g_4) for each type of IoT device. We consider the first group of IoT devices as the historical devices and gradually added the 2nd-4th groups to retrain our graph model following the CL-GraphIoT approach detailed in Section V. This allows our graph model to have continuous learning ability.

To support continuous learning, our CL-GraphIoT adopts two main strategies: parameter sharing and sample replaying. In following contexts, we first evaluate the impact of these strategies on the performance and identify the parameters involved in these strategies.

1) Impact of Parameter Sharing: As described in Table IX, CL-GraphIoT adopts a parameter sharing strategy, where only a small number of GNN layers in the graph learning module and the classification module are retrained when new devices arrive. We vary the number of GNN layers shared by GraphIoT and CL-GraphIoT to study the impact of parameter sharing on performance. GraphIoT has 3 GNN layers, and we test different sharing situations: sharing 1st layer, 2nd layer, 3rd layer, 1st-2nd layers, 2nd-3rd layers, and 1st-2nd-3rd layers. We evaluate the performance and training time under each situation.

As expected, the shared layers of the graph neural network have a significant impact on the performance of CL-GraphIoT. Sharing the 2nd-3rd layers of GraphIoT yields the best performance, with CL-GraphIoT achieving an accuracy of 92.41% and 90.86% on the two datasets, respectively. However, reducing the number of shared layers in the graph neural network improves accuracy at the cost of increased training time overhead. Therefore, to achieve a balance between accuracy and time cost, we choose to share the 2nd-3rd layers of GraphIoT.

As the number of IoT device groups continuously increases, we observe a decrease in accuracy and an increase in training time cost. However, the drop in accuracy is not significant when the third and fourth groups of devices are added, indicating that our algorithm has good continuous learning ability.

TABLE X
IMPACT OF NUMBER OF REPLAYED SAMPLES ON ACCURACY AND TIME OF CL-GRAPHIOT

Number of Replayed	100	200	300	400	500
	TMC_{21}				
Accuracy (ACC)(%)	89.79	90.64	91.34	91.72	**92.41**
Training Time(s)	90.63	102.92	116.35	137.95	**149.67**
	IoT_{18}				
Accuracy (ACC)(%)	75.43	80.27	84.36	88.01	**90.86**
Training Time(s)	74.32	90.52	100.38	112.54	**120.91**

The bold values the highest accuracy, and the shortest training time.

2) Impact of Sample Replaying: To avoid forgetting the characteristics of previously learned devices, CL-GraphIoT adopts a sample replaying strategy. When new types of devices are introduced, we vary the number of replayed samples for each previously learned device to investigate the impact on accuracy. The results are summarized in Table X, which presents the performance only under the arrival of the 3rd group of IoT devices. As expected, we observe that the accuracy of the model increases with an increase in the number of replayed samples for each learned device type, while the training time also increases. To achieve a good balance between accuracy and time cost, we set the number of replayed samples for each learned device type to 500, which results in higher accuracy with moderate time overhead.

E. Ablation Studies

To ascertain the effectiveness of the individual components within our proposed CL-GraphIoT and GraphIoT models, we design several ablation experiments. We examine the performance of these models against their variants described as follows:

- GraphIoT + Fine-tuning: This variant follows all techniques in our GraphIoT, which involve using packet length, time intervals, and subgraph structures as features to learn graph representations. However, when new groups of IoT devices arrive, the model is retrained using only the data of new devices, without employing parameter sharing and sample replaying strategies.
- GraphIoT + Share: This variant follows all techniques in our GraphIoT, which involve using packet length, time intervals, and subgraph structures as features to learn graph

Authorized licensed use limited to: University of Missouri Libraries. Downloaded on August 13,2026 at 05:55:03 UTC from IEEE Xplore. Restrictions apply.

TABLE XI
PERFORMANCE COMPARISON WITH BASELINES

Incremental Times	GraphIoT+Fine-tuning	GraphIoT+Share	GIN+L	GIN+L+T	CL-GraphIoT
	TMC_{21}				
After g1, g2 comes	0.5078	0.5464	0.9269	0.9277	**0.9301**
After g1 and g2, g3 comes	0.1315	0.1576	0.9245	0.9263	**0.9278**
After g1, g2, and g3, g4 comes	0.1285	0.1552	0.9046	0.9095	**0.9241**
	IoT_{18}				
After g1, g2 comes	0.2634	0.2879	0.9771	0.9776	**0.9783**
After g1 and g2, g3 comes	0.1532	0.1778	0.9534	0.9544	**0.9587**
After g1, g2, and g3, g4 comes	0.1476	0.1652	0.8993	0.9064	**0.9086**

The bold values the highest accuracy.

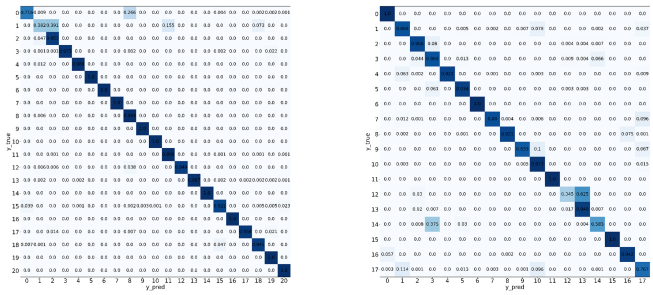
representations. However, when new groups of IoT devices arrive, the 2nd to 3rd GNN layers of the historical model will be shared, and the remaining layers of the model will be retrained using only the data of new devices, employing parameter sharing without sample replaying strategies.

- GIN + L [38]: This variant uses only node features to learn the graph feature and follows most techniques in our GraphIoT. When new groups of IoT devices arrive, this variant employs the same parameter sharing and sample replaying strategies as CL-GraphIoT for continuous learning.
- GIN + L + T: This variant uses both node and edge features to learn the graph feature and follows most techniques in our GraphIoT. When new groups of IoT devices arrive, this variant employs the same parameter sharing and sample replaying strategies as CL-GraphIoT for continuous learning.

As indicated in Table XI, all models demonstrate a decline in accuracy with the increase in the number of IoT device types. However, CL-GraphIoT exhibits a more gradual decrease, signifying its robustness and advanced handling of new device types. Remarkably, CL-GraphIoT consistently achieves the highest accuracy across all scenarios, highlighting its superior performance. This further demonstrates that CL-GraphIoT can continuously learn the features of new device flows and maintain effective detection performance in open IoT environments.

Conversely, the GraphIoT + Fine-tuning and GraphIoT + Share models perform the poorest, primarily due to catastrophic forgetting when trained only with data from new devices. In contrast, the sample replaying strategy in CL-GraphIoT effectively addresses this issue. Moreover, while GIN+L and GIN+L+T models learn graph features considering node and edge characteristics, CL-GraphIoT additionally leverages subgraph features, which contributes to its enhanced performance.

In Fig. 12, the confusion matrices for the CL-GraphIoT method are depicted, showcasing its performance on the TMC_{21} and IoT_{18} datasets. The results illustrate robust classification capabilities, with 17 IoT device types achieving an accuracy over 0.95, and 95% of the device types achieving an accuracy over 0.7 on the TMC_{21} dataset. Similarly, on the IoT_{18} dataset, 11 IoT device types reach an accuracy over 0.9, and 89% of the device types maintain an accuracy over 0.7. These performance metrics demonstrate the efficacy of CL-GraphIoT in managing the challenges posed by a continuously expanding array of IoT devices. The method benefits from an advanced end-to-end learning architecture, augmented by a parameter sharing and sample replaying mechanism, which collectively enhance its capability to adapt to new device types while sustaining high accuracy levels over time.



(a) TMC_{21} (b) IoT_{18}

Fig. 12. Confusion matrix of CL-GraphIoT

TABLE XII
IMPACT OF FIVE INCREMENTS ROUNDS ON THE ACCURACY AND TIME OF CL-GRAPHIOT(ADD 3 NEW DEVICE TYPES/ROUND)

Incremental Rounds	1	2	3	4	5
	TMC_{21}				
Accuracy (ACC)(%)	94.35	93.29	93.22	92.36	91.49
Training Time(s)	51.28	62.34	85.54	96.07	118.21
	IoT_{18}				
Accuracy (ACC)(%)	98.31	95.23	92.54	91.11	90.71
Training Time(s)	20.95	43.98	57.44	76.60	86.50

To further validate the effectiveness of CL-GraphIoT, we conducted 5 incremental rounds to investigate its impact on accuracy and training time. Specifically, three new IoT device types were added in each round. As depicted in Table XII, although there was a decline in accuracy with an increasing number of IoT device types, CL-GraphIoT demonstrated a relatively gradual decrease. Additionally, the increase in training time was modest. These findings suggest that CL-GraphIoT can continuously learn the features of new IoT device flows and maintain high detection performance in open IoT environments, further validating its effectiveness.

VII. CONCLUSION

This paper proposes GraphIoT, a lightweight IoT device detection method based on graph classifiers that utilizes lightweight flow information to create an IoT Device Traffic Graph Representation (IoT-DTGR). The proposed method turns the IoT device detection problem into a graph classification problem and employs a powerful Graph Neural Network that considers both node and edge features, as well as subgraph structures of IoT-DTGRs, to classify graphs and thus identify devices. GraphIoT primarily learns graph features through the

Authorized licensed use limited to: University of Missouri Libraries. Downloaded on August 13,2026 at 05:55:03 UTC from IEEE Xplore. Restrictions apply.

iterative aggregation of neighbor node features, edge features, and subgraph structures. Through n-layer iterations, GraphIoT can capture the local and global relationships within the n-hop network neighborhood of IoT-DTGRs. The paper also introduces the incremental learning framework called CL-GraphIoT, which continuously learn the features of new IoT device flows and maintain high detection performance in open IoT environments. We call our solution a lightweight device detection algorithm for two reasons. First, it only uses lightweight flow information such as packet length, timestamp, and direction to detect devices. Second, GraphIoT only uses 3 GNN layers to achieve high detection accuracy, resulting in low computation complexity and fast detection speed. The performance of GraphIoT and CL-GraphIoT is evaluated on two datasets, and the results demonstrate that the proposed method outperforms state-of-the-art methods, making it a promising solution for IoT device identification and security.

REFERENCES

[1] W. Zhou, Y. Jia, A. Peng, Y. Zhang, and P. Liu, "The effect of IoT new features on security and privacy: New threats, existing solutions, and challenges yet to be solved," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 1606–1616, Apr. 2019.

[2] J. François, H. Abdelnur, R. State, and O. Festor, "Automated behavioral fingerprinting," in *Proc. Recent Adv. Intrusion Detection: 12th Int. Symp.*, Saint-Malo, France, Springer, 2009, pp. 182–201.

[3] J. François, H. Abdelnur, R. State, and O. Festor, "Machine learning techniques for passive network inventory," *IEEE Trans. Netw. Service Manag.*, vol. 7, no. 4, pp. 244–257, Dec. 2010.

[4] K. Gao, C. Corbett, and R. Beyah, "A passive approach to wireless device fingerprinting," in *Proc. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2010, pp. 383–392.

[5] T. Kohno, A. Broido, and K. Claffy, "Remote physical device fingerprinting," *IEEE Trans. Dependable Secure Comput.*, vol. 2, no. 2, pp. 93–108, Second Quarter, 2005.

[6] S. V. Radhakrishnan, A. S. Uluagac, and R. Beyah, "GTID: A technique for physical device and device type fingerprinting," *IEEE Trans. Dependable Secure Comput.*, vol. 12, no. 5, pp. 519–532, Sep./Oct. 2015.

[7] S. Siby, R. R. Maiti, and N. O. Tippenhauer, "IoTScanner: Detecting and classifying privacy threats in IoT neighborhoods," 2017, *arXiv: 1701.05007*. [Online]. Available: http://arxiv.org/abs/1701.05007

[8] H. Guo and J. Heidemann, "IP-based IoT device detection," in *Proc. Workshop IoT Secur. Privacy*, New York, NY, USA, 2018, pp. 36–42. [Online]. Available: https://doi.org/10.1145/3229565.3229572

[9] M. Swarnkar and R. Kumar, "BitIoT: A bit level deep packet inspection method for identification of MQTT-based IoT devices in the wild," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 3, pp. 2866–2875, Jun. 2024.

[10] A. Acar et al., "Peek-a-boo: I see your smart home activities, even encrypted!," in *Proc. 13th ACM Conf. Secur. Privacy Wireless Mobile Netw.*, New York, NY, USA, 2020, pp. 207–218. [Online]. Available: https://doi.org/10.1145/3395351.3399421

[11] J. Bao, B. Hamdaoui, and W.-K. Wong, "IoT device type identification using hybrid deep learning approach for increased IoT security," in *Proc. Int. Wireless Commun. Mobile Comput.*, 2020, pp. 565–570.

[12] S. Marchal, M. Miettinen, T. D. Nguyen, A.-R. Sadeghi, and N. Asokan, "AuDi: Toward autonomous IoT device-type identification using periodic communication," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 6, pp. 1402–1412, Jun. 2019.

[13] H. Cheng, W. Dong, Y. Zheng, and B. Lv, "Identify IoT devices through web interface characteristics," in *Proc. IEEE 6th Int. Conf. Comput. Commun. Syst.*, 2021, pp. 405–410.

[14] F. Sawadogo, J. Violos, A. Hameed, and A. Leivadeas, "An unsupervised machine learning approach for IoT device categorization," in *Proc. IEEE Int. Mediterranean Conf. Commun. Netw.*, 2022, pp. 25–30.

[15] M. Miettinen, S. Marchal, I. Hafeez, N. Asokan, A.-R. Sadeghi, and S. Tarkoma, "IoT sentinel: Automated device-type identification for security enforcement in IoT," in *Proc. IEEE 37th Int. Conf. Distrib. Comput. Syst.*, 2017, pp. 2177–2184.

[16] R. Perdisci, T. Papastergiou, O. Alrawi, and M. Antonakakis, "IoTFinder: Efficient large-scale identification of IoT devices via passive DNS traffic analysis," in *Proc. IEEE Eur. Symp. Secur. Privacy*, 2020, pp. 474–489.

[17] S. Roy, J. Li, B.-J. Choi, and Y. Bai, "A lightweight supervised intrusion detection mechanism for IoT networks," *Future Gener. Comput. Syst.*, vol. 127, pp. 276–285, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167739X21003733

[18] S. Zhang et al., "Unsupervised IoT fingerprinting method via variational auto-encoder and K-means," in *Proc. IEEE Int. Conf. Commun.*, 2021, pp. 1–6.

[19] A. Sivanathan et al., "Classifying IoT devices in smart environments using network traffic characteristics," *IEEE Trans. Mobile Comput.*, vol. 18, no. 8, pp. 1745–1759, Aug. 2019.

[20] M. Skowron, A. Janicki, and W. Mazurczyk, "Traffic fingerprinting attacks on Internet of Things using machine learning," *IEEE Access*, vol. 8, pp. 20386–20400, 2020.

[21] N. Yousefnezhad, A. Malhi, and K. Främling, "Automated IoT device identification based on full packet information using real-time network traffic," *Sensors*, vol. 21, no. 8, 2021, Art. no. 2660. [Online]. Available: https://www.mdpi.com/1424--8220/21/8/2660

[22] Y. Wan, K. Xu, F. Wang, and G. Xue, "IoTAthena: Unveiling IoT device activities from network traffic," *IEEE Trans. Wireless Commun.*, vol. 21, no. 1, pp. 651–664, Jan. 2022.

[23] A. Chawla, P. Babu, T. Gawande, E. Aumayr, P. Jacob, and S. Fallon, "Intelligent monitoring of IoT devices using neural networks," in *Proc. 24th Conf. Innov. Clouds, Internet Netw. Workshops*, 2021, pp. 137–139.

[24] I. Priyadarshini et al., "Survivability of industrial Internet of Things using machine learning and smart contracts," *Comput. Elect. Eng.*, vol. 107, 2023, Art. no. 108617. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0045790623000423

[25] M. Lopez-Martin, B. Carro, A. Sanchez-Esguevillas, and J. Lloret, "Network traffic classifier with convolutional and recurrent neural networks for Internet of Things," *IEEE Access*, vol. 5, pp. 18042–18050, 2017.

[26] J. Ortiz, C. Crawford, and F. Le, "DeviceMien: Network device behavior modeling for identifying unknown IoT devices," in *Proc. Int. Conf.Internet Things Des. Implementation*, New York, NY, USA, 2019, pp. 106–117. [Online]. Available: https://doi.org/10.1145/3302505.3310073

[27] R. R. Chowdhury, A. C. Idris, and P. E. Abas, "A deep learning approach for classifying network connected IoT devices using communication traffic characteristics," *J. Netw. Syst. Manage.*, vol. 31, no. 1, 2023, Art. no. 26.

[28] Y. Jin, J. Zhou, and Y. Gao, "HSGAN-IoT: A hierarchical semi-supervised generative adversarial networks for IoT device classification," *Comput. Netw.*, vol. 243, 2024, Art. no. 110299. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1389128624001312

[29] Z. Solatidehkordi, J. Ramesh, A. Al-Ali, A. Osman, and M. Shaaban, "An IoT deep learning-based home appliances management and classification system," *Energy Rep.*, vol. 9, pp. 503–509, 2023.

[30] Y. Luo, X. Chen, N. Ge, W. Feng, and J. Lu, "Transformer-based device-type identification in heterogeneous IoT traffic," *IEEE Internet Things J.*, vol. 10, no. 6, pp. 5050–5062, Mar. 2023.

[31] V. Rey, P. M. S. Sánchez, A. H. Celdrán, and G. Bovet, "Federated learning for malware detection in IoT devices," *Comput. Netw.*, vol. 204, 2022, Art. no. 108693.

[32] L. Fan et al., "AutoIoT: Automatically updated IoT device identification with semi-supervised learning," *IEEE Trans. Mobile Comput.*, vol. 22, no. 10, pp. 5769–5786, Oct. 2023.

[33] H. M. Zahid et al., "A framework for identification and classification of IoT devices for security analysis in heterogeneous network," *Wireless Commun. Mobile Comput.*, vol. 2022, no. 1, 2022, Art. no. 8806184.

[34] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Accurate decentralized application identification via encrypted traffic analysis using graph neural networks," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2367–2380, 2021.

[35] Y. Li, "Research and application of deep learning in image recognition," in *Proc. IEEE 2nd Int. Conf. Power, Electron. Comput. Appl.*, 2022, pp. 994–999.

[36] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, "Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing," *ACM Comput. Surv.*, vol. 55, no. 9, pp. 1–35, Jan. 2023. [Online]. Available: https://doi.org/10.1145/3560815

[37] N. Shervashidze, P. Schweitzer, E. J. van Leeuwen, K. Mehlhorn, and K. M. Borgwardt, "Weisfeiler-lehman graph kernels," *J. Mach. Learn. Res.*, vol. 12, no. 9, pp. 2539–2561, Nov. 2011.

Authorized licensed use limited to: University of Missouri Libraries. Downloaded on August 13,2026 at 05:55:03 UTC from IEEE Xplore. Restrictions apply.

[38] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?," 2018, *arXiv: 1810.00826*. [Online]. Available: http://arxiv.org/abs/1810.00826
[39] A. Wijesinghe and Q. Wang, "A new perspective on "how graph neural networks go beyond weisfeiler-lehman?"," in *Proc. Int. Conf. Learn. Representations*, 2022, pp. 1–23. [Online]. Available: https://openreview.net/forum?id=uxgg9o7bI_3
[40] IoT traffic traces. 2018, [Online]. Available: https://iotanalytics.unsw.edu.au/
[41] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, New York, NY, USA, 2018, pp. 1928–1943. [Online]. Available: https://doi.org/10.1145/3243734.3243768
[42] V. Rimmer, D. Preuveneers, M. Juárez, T. van Goethem, and W. Joosen, "Automated feature extraction for website fingerprinting through deep learning," 2017, *arXiv: 1708.06376*. [Online]. Available: http://arxiv.org/abs/1708.06376
[43] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," 2017, *arXiv: 1710.10903*.



Yansong Yin is currently working toward the PhD degree with the College of Computer Science and Electronics Engineering, Hunan University. His research interests include traffic classification and IoT device detection.



Kun Xie (Member, IEEE) received the PhD degree in computer application from Hunan University, Changsha, China, in 2007. She is currently a professor with Hunan University. She has published mroe than 60 articles in major journals and conference proceedings, including *IEEE/ACM Transactions on Networking*, *IEEE Transactions on Mobile Computing*, *IEEE Transactions on Computers*, *IEEE Transactions on Wireless Communications*, *IEEE Transactions on Services Computing*, SIGMOD, INFOCOM, ICDCS, SECON, DSN, and IWQoS. Her research interests include network measurement, network security, Big Data, and AI.



Shiming He received the BS degree in information security, and the PhD degree in computer science and technology from Hunan University, China, in 2006 and 2013, respectively. She is currently an associated professor with the School of Computer and Communication Engineering, Changsha University of Science and Technology, Changsha, China. Her research interests include machine learning, data analysis, and anomaly detection.



Yanbiao Li (Member, IEEE) received the BS degree in mathematics and the PhD degree in computer science from Hunan University, in 2009 and 2016, respectively. He is currently a professor with the Computer Network Information Center (CNIC), Chinese Academy of Sciences (CAS), and the University of Chinese Academy of Sciences (UCAS). His research interests include networked systems, packet processing algorithms and routing securit.



Jigang Wen received the PhD degree in computer applications from Hunan University, China, in 2011. He worked as a research assistant with the Department of Computing, The Hong Kong Polytechnic University from 2008 to 2010. He also worked as a postdoctoral fellow with the Institute of Computing Technology Chinese Academy of Sciences from 2012 to 2017. Currently, he is an associate professor with the School of Computer Science and Engineering, Hunan University of Science and Technology. His research interests include measurement and management in IoT and high-speed networks.



Zulong Diao received the PhD degree in software engineering from Hunan University, Changsha, China, in 2019. He is currently an associate professor with the Network Technology Research Center, Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China. His research interests include machine learning, traffic analysis and Internet measurement.



Dafang Zhang received the PhD degree in application mathematics from Hunan University, Changsha, China, in 1997. He is currently a professor with Hunan University, Changsha, China. His research interests include packet processing, Internet measurement, wireless network and mobile computing, and Big Data.



Gaogang Xie received the PhD degrees in computer science from Hunan University, China, in 2002. He is currently a professor with Computer Network Information Center (CNIC), Chinese Academy of Sciences (CAS), and the University of Chinese Academy of Sciences (UCAS). His research interests include Internet architecture, packet processing and forwarding, and Internet measurement.

Authorized licensed use limited to: University of Missouri Libraries. Downloaded on August 13,2026 at 05:55:03 UTC from IEEE Xplore. Restrictions apply.